

# Clase 308 — Capstone: campaña de bug bounty

Parte: **16 — Capstones y preparación de certificaciones** · Fuente: *OWASP Web Security Testing Guide · Bug Bounty Bootcamp (Vickie Li)* 🕒 Duración estimada: **150 min** · Nivel: **Avanzado**

## 🎯 Objetivo

Ejecutar una **campaña de bug bounty ética y estructurada**: elegir un programa, leer y respetar su alcance, hacer reconocimiento eficiente, priorizar vectores de alto impacto, validar hallazgos con PoC reproducibles y **redactar reportes aceptables** que maximicen la probabilidad de triage positivo. Integra la Parte 6 (web/OWASP), la Parte 7 (recon) y la Parte 3 (redes), aplicándolas al mundo real dentro de un marco legal.

⚠️ **Ética y legalidad**: solo actúa dentro del **alcance (scope)** de un programa que te autoriza explícitamente. Probar fuera de alcance, exfiltrar datos reales o pivotar sin permiso es delito y viola las reglas del programa. Lee el *safe harbor* antes de empezar.

## 📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Seleccionar** un programa y comprender su alcance y reglas.
- 2. **Automatizar** un flujo de reconocimiento respetuoso con el scope.
- 3. **Priorizar** vectores por impacto y probabilidad de recompensa.
- 4. **Validar** un hallazgo con PoC mínima y no destructiva.
- 5. **Redactar** un reporte con impacto, pasos de reproducción y remediación.

## 📖 Temas

| # | Tema                                | Por qué importa                     |
|---|-------------------------------------|-------------------------------------|
| 1 | Plataformas y programas             | HackerOne, Bugcrowd, Intigriti, VDP |
| 2 | Alcance y safe harbor               | Límite legal de la actividad        |
| 3 | Reconocimiento (subdominios/assets) | Superficie donde buscar             |
| 4 | Vectores de alto valor              | IDOR, SSRF, XSS, auth flaws         |
| 5 | PoC no destructiva                  | Demostrar sin causar daño           |
| 6 | Reporte y CVSS                      | Comunicar impacto con claridad      |
| 7 | Duplicados y triage                 | Gestionar expectativas              |

## Definiciones y características

- **Scope:** activos y pruebas permitidas por el programa. *Característica:* fuera de él, todo está prohibido.
- **Safe harbor:** cláusula que protege al investigador que actúa de buena fe dentro del scope. *Característica:* define tu cobertura legal.
- **VDP (Vulnerability Disclosure Program):** programa sin recompensa monetaria. *Característica:* bueno para practicar reputación.
- **PoC (Proof of Concept):** demostración mínima del bug. *Característica:* reproducible y sin impacto real.
- **Triage:** revisión del reporte por el equipo del programa. *Característica:* decide validez, severidad y duplicado.
- **Duplicado:** bug ya reportado por otro. *Característica:* no se recompensa; la velocidad importa.

## Herramientas y preparación

- Cuenta en una plataforma (HackerOne/Bugcrowd/Intigriti) y lectura del scope de un programa.
- Recon: `subfinder`, `amass`, `httpx`, `nuclei`, `gau` / `waybackurls`, `ffuf`.
- Proxy: **Burp Suite** (Community/Pro) para interceptar y manipular.
- Notas y capturas para el reporte; plantilla de reporte con secciones estándar.
- Un laboratorio propio (PortSwigger Web Security Academy) para **practicar** técnicas antes de ir al programa real.

## Laboratorio guiado

Practica las técnicas en PortSwigger Academy o tu laboratorio; aplícalas en un programa **solo dentro de su scope**.

1. **Elige programa y lee el scope.** Anota activos en scope, exclusiones, tipos de bug aceptados y el safe harbor.
2. **Reconocimiento pasivo.** Enumera subdominios con `subfinder` / `amass` y resuélvelos con `httpx` (respetando scope):

```
bash subfinder -d ejemplo-en-scope.com -silent | httpx -silent -status-code -title
```

3. **Mapea la superficie.** Recolecta URLs históricas con `gau` / `waybackurls` y clasifícalas por funcionalidad (login, API, upload).
4. **Escaneo ligero.** Corre `nuclei` con plantillas no intrusivas para detectar exposiciones conocidas.
5. **Prioriza vectores.** Elige 2–3 clases de alto valor coherentes con la app: IDOR, SSRF, XSS almacenado, fallos de autorización.
6. **Prueba manual con Burp.** Intercepta, manipula parámetros y objetos; busca acceso a recursos ajenos (IDOR) o inyecciones.
7. **Valida con PoC mínima.** Demuestra el bug sin exfiltrar datos reales (usa tu propia cuenta/objeto de prueba).
8. **Puntúa.** Asigna CVSS y describe el impacto de negocio.
9. **Redacta el reporte.** Título claro, resumen, pasos de reproducción numerados, PoC, impacto y remediación.
10. **Envía y gestiona.** Responde al triage con profesionalidad; acepta duplicados sin frustración.

## Ejercicios

1. Resume el scope de un programa real en una tabla (in/out, tipos aceptados).
2. Ejecuta un flujo de recon y lista 10 activos en scope.
3. Reproduce un IDOR y un XSS en PortSwigger Academy y documenta el PoC.
4. Redacta un reporte completo de un bug de laboratorio.
5. Puntúa dos hallazgos con CVSS y justifica la diferencia.
6. Reescribe un reporte pobre para hacerlo "triable".

## Reto verificable

Entrega un **reporte de vulnerabilidad** ( `reporte-bugbounty.md` ) de calidad profesional —basado en un hallazgo de laboratorio (PortSwigger/tu app) o, si aplica, de un programa dentro de scope— con: resumen, activo afectado, pasos de reproducción numerados, PoC no destructiva, CVSS e impacto de negocio y remediación. Incluye una nota de scope/ética.

**Criterio de aceptación:** los pasos de reproducción los puede seguir un tercero sin ambigüedad, la PoC no es destructiva ni exfiltra datos reales, el CVSS es coherente con el impacto, y hay una declaración explícita de que se actuó dentro de scope.

## Errores comunes

| Síntoma / mensaje                | Causa y cómo arreglar                                               |
|----------------------------------|---------------------------------------------------------------------|
| "Me banearon del programa"       | Probaste fuera de scope; lee y respeta el alcance siempre.          |
| "Marcaron mi bug como duplicado" | Alguien reportó antes; mejora recon y velocidad, no te desanimes.   |
| "Rechazaron por 'informativo'"   | Falta impacto demostrable; refuerza el PoC y el impacto de negocio. |
| "El reporte no se entiende"      | Pasos vagos; numera cada acción y añade capturas.                   |
| "nuclei tumbó el servicio"       | Escaneo intrusivo; usa plantillas suaves y respeta rate limits.     |

## Preguntas frecuentes

**? ¿Necesito Burp Pro?** No para empezar. Burp Community y las herramientas abiertas bastan para muchos hallazgos.

**? ¿Puedo exfiltrar datos para probar el impacto?** No. Demuestra el acceso con un objeto propio o una prueba mínima; nunca extraigas datos reales de terceros.

**? ¿Dónde practico sin riesgo legal?** PortSwigger Web Security Academy y laboratorios propios. Aplica en programas reales solo dentro de scope.

**? ¿Cómo evito duplicados?** Mejor recon, vectores menos obvios y rapidez. Aun así, los duplicados son parte del juego.

## Referencias

---

- OWASP WSTG: <https://owasp.org/www-project-web-security-testing-guide/>
- PortSwigger Web Security Academy: <https://portswigger.net/web-security>
- HackerOne: <https://www.hackerone.com/> · Bugcrowd: <https://www.bugcrowd.com/>
- Vickie Li, *Bug Bounty Bootcamp* (No Starch Press).
- ProjectDiscovery (subfinder/httpx/nuclei): <https://github.com/projectdiscovery>

## Siguiente clase

---

Clase 309 - Construccion de portafolio y home lab permanente